Image Text Enhancer: A Lightweight, Easy-to-Use, and Efficient C++ Library

Algorithm Engineering 2026 Project

Daniel Motz
Friedrich Schiller University
Jena, Germany
daniel.motz@uni-jena.de

Leonard Teschner
Friedrich Schiller University
Jena, Germany
leonard.teschner@uni-jena.de

Mher Mnatsakanyan
Friedrich Schiller University
Jena, Germany
mher.mnatsakanyan@uni-jena.de

Abstract

TBC

Keywords

task parallelism, WebAssembly, OpenMP, image processing, C++

ACM Reference Format:

Daniel Motz, Leonard Teschner, and Mher Mnatsakanyan. 2026. Image Text Enhancer: A Lightweight, Easy-to-Use, and Efficient C++ Library: Algorithm Engineering 2026 Project. In *Proceedings of AEPRO 2026: Algorithm Engineering Projects (AEPRO 2026).* ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

RQ Can a lightweight document enhancement pipeline be optimised to achieve strong multicore performance and practical WebAssembly deployment while preserving the enhancement behaviour of the baseline implementation?

RQ1. How much faster is the optimised pipeline than the baseline on native CPU execution?
RQ2. How well does the optimised pipeline scale under strong and runtime scaling from 1 to 8 cores?
RQ3. What runtime overhead does the WebAssembly version introduce compared to native execution, and is that overhead still acceptable for practical use?

1.1 Background

The extraction of textual information from images — from scanned books to even receipts captured by smartphone cameras — have become an important part of everyday life [1, 8]. Any such extraction necessarily begins with the pre-processing of recorded images through techniques such as binarisation, noise removal, and geometric transformation, a process generally referred to as *document enhancement.* Numerous tools focused on this niche have been proposed. Although these approaches feature broad capabilities, they are often associated with substantial complexity on the user's end and entail considerable software dependencies. These dependencies restrict applicability, particularly in settings where binary size is constrained, such as mobile and web platforms. This limitation assumes added importance in a context where most digital media interactions now take place on smartphones and application development is increasingly oriented toward edge-computing models. Consequently, efficient and platform-independent implementations that exploit parallelisation and other optimisation techniques are of considerable importance for reducing processing time and enhancing user satisfaction.

Table 1: Comparison of related tools.

Tool	Lightweight	Licence	Commercial
ImageMagick	✗	Own	✓
Leptonica	✓	Own	✓
Unpaper	✗	Mixed	✗

1.2 Related Work

Image enhancement and document preprocessing are covered by a wide range of open-source tools and commercial software. The scope ranges from general purpose image processing and OCR libraries to specialised tools for post-processing scanned documents.

ImageMagick [4] is a suite for image processing that is available as a command line tool and as a library. It offers functions such as format conversion, resizing, filtering, and colour correction. However, ImageMagick is not specifically designed to enhance scanned or photographed images. Similar to OpenCV, the diverse features result in a large code base.

The C-based Leptonica library [3] is a general-purpose image processing and image analysis application. It is a dependency to both Tesseract and OpenCV. Although Leptonica is relatively lightweight compared to the other libraries, much like our approach, it primarily serves as a low-level library and does not offer ready-to-use, configurable processing pipelines.

A very similar tool to our approach is Unpaper [5], which focuses on post-processing scanned documents. It offers features such as deskewing, binarisation, and noise reduction that have been developed specifically for this purpose. The tool also offers a CLI variant, with which pipeline steps can be configured and disabled. Unpaper requires the library ffmpeg, which in the standard configuration consumes >100MiB of space.

1.3 Our Contribution

Unlike the existing tools and libraries described, our approach is designed to be minimalistic, extensible, and user-friendly. Instead of being a comprehensive image processing library or OCR engine, we focus on a lightweight, modular image enhancement pipeline, focused on printed and handwritten text, written entirely in C++ with only CImg as a dependency, which is itself is lightweight

has no further dependencies [7]. Image Text Enhancer (ITE) has a lightweight build footprint, requiring only three build dependencies: OpenMP, X11, and JPEG. It can be imported as a library and also as a CLI executable—with 1.7 MiB on Linux.

Compared to OpenCV and ImageMagick, our work is characterised by its deliberate focus on document enhancement and compactness. Unlike Leptonica, our pipeline is specialised in the post-processing of scanned documents and unlike Unpaper, our binary including dependencies is ≈50× smaller. In addition, our licensing model (MIT-0 and CeCILL-C) permits use in research and commercial applications. By combining minimal dependencies, a modular architecture, portability through C++ and OpenMP, and a commercially usable licence, our pipeline fills a gap between large-scale, general-purpose image processing libraries and highly specialised tools for document enhancement.

Furthermore, we systematically optimise and improve our baseline implementation and evaluate the performance gains through various experiments. In a proof of concept, we show that our approach can be executed on WASM platforms, with only minor performance drawbacks.

1.4 Outline

In section 2, we introduce the pipeline that we will implement and optimise. In section 3, we describe the optimisation techniques used. These are evaluated in section 4, where we compare the performance of the pipeline under different settings to the baseline implementation. Finally, we provide an outlook on possible future work 5.

2 Pipeline

The pipeline is designed to be modular, enabling users to activate, deactivate and configure each stage as required, both through the CLI and the library. Some stages can intentionally not be disabled. These exceptions are marked in algorithm 1 with an **asterisk**.

Algorithm 1 Image Text Enhancer Pipeline

(1) Grayscale Conversion*
(2) Deskew
(3) Contrast Enhancement
(4) Denoise: (Adaptive) Gaussian and Median Blur
(5) Binarisation*: Otsu, Sauvola, Bataineh
(6) Morphological Operations
(7) Colour passthrough

The user can choose between different algorithms for certain stages—as these have different effects on the result, users can select the algorithm best suited to their application. For instance, Bataineh's adaptive binarisation method with automatic window adjustment [2] is helpful when binarising historical, handwritten documents with uneven writing and lighting, whereas Otsu's method [6] is a good choice for binarising typewritten documents with uniform font and lighting. Standard configurations are provided for each step, enabling the pipeline to be used straight away. These configurations deliver good results for most use cases and are based on fixed recommendations from the authors. This means that the pipeline can be applied immediately. However, users can also customise the configurations to optimise the pipeline and improve their specific results.

We provide a command-line application that can be used independently of a program to access the pipeline. This enables the pipeline to be configured and used directly via the command line.

3 Optimisations

We hypothesise that the baseline implementation fails to scale linearly with problem size, although such scaling would be desirable for image processing workloads. We therefore concentrated on identifying low-hanging fruits at the level of high-level algorithmic design, for example by selecting more efficient algorithms for specific tasks, while aiming to adhere to best practices such as avoiding false sharing and writing code amenable to auto-vectorisation. For the purposes of the paper, we focus on the stages, whose optimisations had the biggest effect. The full list of applied optimisations is documented in the project's repository.

3.1 Deskew

Instead of using a "rotate-and-project" approach, which repeatedly rotates the image to find an appropriate angle, we applied the Radon transform, which gives an acceptable rotation after the first application. Also, we removed an overlook, where both the deskew for the grayscale and colour image were calculated, if the colour passthrough option was enabled.

3.2 Bataineh

The main feature and bottleneck of the Bataineh algorithm is the adaptive sliding window, which takes into account the number of pixels from three categories—background, foreground and confusion [2]. This requires summing over areas of the image with masks reflecting these categories. The baseline approach was to sum using a nested for loop for each window. The optimised variant uses two integral image for background and confusion pixels, yielding a significant complexity reduction. We estimate our baseline implementation with $O(n \cdot w^2)$—where n is the number of pixels and w the window size—while the improved implementation is in $O(n)$.

3.3 Grayscale

3.4 Morphology

The pipeline only supports dilation and erosion, which can both be realised by linear filters. The baseline implementation used four nested for-loops[1] for iterating the image and scanning the neighbourhood. Since the filter is separable, it is divided into a horizontal and vertical pass. Instead of scanning the neighbourhood, a queue is used to compute the min/max of a given 1D window in amortised $O(1)$.

3.5 Loop level Optimisations

To reduce the runtime of pixel processing loops, logical optimisations are made to the main loops that iterate over the pixels. In contrast enhancement, for example, the linear stretching formula

[1] for colour images it is five, as it is done for each channel, but our approach is only used on binary images

is calculated in advance and stored in a lookup table. As a result, the main loop performs an $O(1)$ query per loop iteration instead of floating-point calculations. In addition, the if/else branch in the pixel processing loop is replaced by std::min clamping. This allows the compiler to auto-vectorize the loop with SIMD. In Sauvola binarisation, a branchless lookup is also implemented to improve performance.

4 Experiments

To evaluate the performance of the pipeline with the optimisations, various experiments are conducted and the results are compared with the baseline implementation. The experiments examine the runtime when processing images of different sizes with and without parallelisation.

The experiments assess three aspects: (1) single-threaded and multi-threaded runtime scaling, (2) strong-scaling behaviour and (3) runtime scaling of the singular steps in the pipeline.

4.1 Setup

All stages of the pipeline are enabled, with Bataineh binarisation and the adaptive Gaussian and adaptive median filters selected.

For our measurements, we use a machine with an AMD Ryzen 9950X3D 16-core processor (32 hyperthreads) running WSL2 with Ubuntu 24.04.1 LTS with 64 GB of RAM[2]. Each core has a base frequency of 4.3 GHz, with boost frequency of around 5.5 GHz. Care has been taken to limit the number of background processes, so as not to affect performance.

We selected a small but representative set of document images spanning both document type and resolution. The benchmark includes textual documents of various sizes and degrees of detail (see Table **??** for details). These choices aim to capture common document-enhancement scenarios while also testing scalability to very large inputs.

4.2 Results

Figure 1 shows the development of runtime with increasing image resolution for the baseline and optimised implementations. Unsurprisingly, the runtime t increases with the problem size s. We found, that the optimised implementation scales almost linear, surpassing the baseline. For $c_1, c_2 < 1.7 \cdot 10^{-5}$ we observe

$$t_{\text{baseline}} \propto c_1 \cdot s^{1.660}$$
$$t_{\text{optimised}} \propto c_2 \cdot s^{1.071}$$

The results for runtime scaling are seemingly good. We observe, that the optimisations lead to a significant improvement of the runtime, which is consistent with growing input size. We observe, that our hope of the program having an almost linear with growing problem size was fulfilled, with some overhead involved. In any parallel program, there is some necessary overhead involved, in the form of e.g. I/O, data distribution or synchronisation. However, the runtime scaling benchmark however does not tell us about the use of parallel resources. For this, we use a strong scaling benchmark, which runs

[2]It is clear that this comes with a performance penalty, however we can expect an even additive penalty across our conducted tests. See this blog post which tested the penalty on very similar hardware.

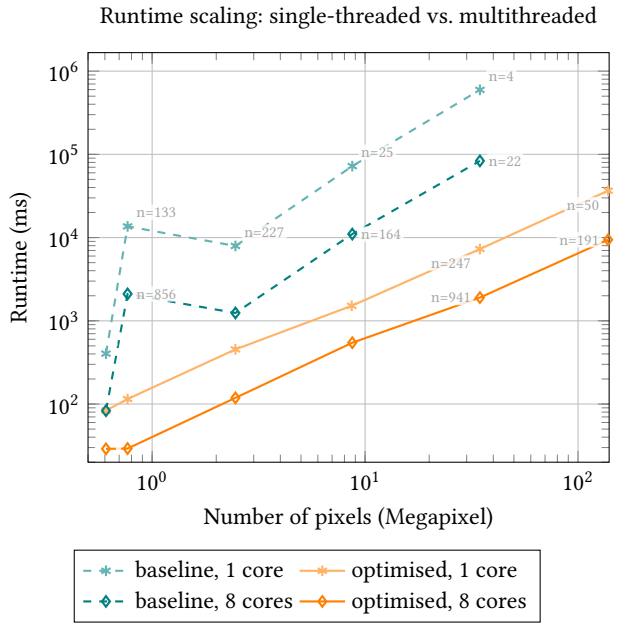

Figure 1: Runtime as a function of image size for the baseline and optimised implementations in single-threaded and 8-core execution. Dashed lines indicate 1-core measurements, solid lines indicate 8-core measurements. Labels are shown only for measurements whose sample count differs from the standard case of 1000 repetitions. For clarity, all but one documents with image size of $\approx 10^1$ Megapixels were omitted, as the results were representative for each other.

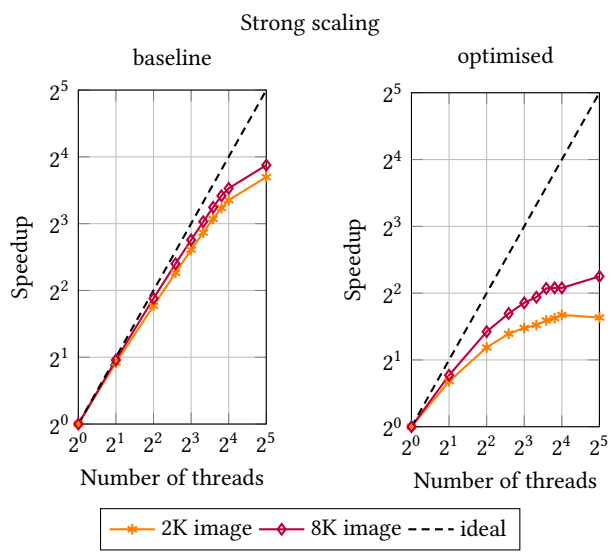

Figure 2: Strong scaling behaviour of the baseline and optimised implementations. Speedup is shown as a function of thread count for two input images (2K and 16K). The dashed black line indicates ideal linear scaling. Each point in the plot represents the mean of 50 trials.

a benchmark on a fixed problem size with an increasing number of threads allocated to the program, giving us information about how much faster the program gets with more parallel resources. On our machine, this involved benching in serial, all the way up to 32 threads. In Figure 2 we observe a well-behaved speedup of the baseline implementation, whereas the optimised code scales worse, where a 32-fold thread increase results in roughly 4-fold speedup. This usually indicates, that there are serial regions in the program, that dominate after the optimisations have been applied, or, that the CPU speed is bound by waiting on e.g. main memory. It is also expected, that the scaling is better for images with a larger problem size, namely because unavoidable serial regions like I/O take up less of the program's execution time. However, from this information alone, we cannot conclude a meaningful cause, but a profiling of the baseline and optimised program and the runtimes of the individual stages would be more conclusive. Table 2 shows two main time improvements, namely, **Deskew** and **Binarisation (Bataineh)** which are individually 98% faster. From the profiling of the baseline implementation we see (todo: insert appropriate figure somewhere) that the Bataineh algorithm makes good use of parallel resources, i.e. the summing of the area is highly parallelised. The binarisation also dominates the baseline program runtime, with about 95%. This explains the good strong scaling performance and also why the optimised variant is so much quicker.

The new bottleneck now becomes memory intialisation. When looking for hotspots in the program we noticed, that there were plenty of serial regions in the program. After further investigation and looking at the assembly code, we realised that most of the serial time is being spent on memory operations, which include loading and writing the image and the allocation of stage-local scratch buffers, e.g. for erosion and dilation or even an output image for the binarisation functions. Roughly 6 seconds of the total pipeline time was spent on such memory operations, hugely limiting the impact any parallelisation effort can have.

The results of the individual phases show a significant reduction in runtime in the deskewing and Bataineh binarisation phases when comparing the baseline and optimised implementations. Table 2 shows the runtime of the individual phases for the KITA test image with a resolution of 3840x2254 on 8 cores. It can be seen that the optimisations lead to a massive reduction in phase runtime of over 98% for Bataineh binarisation in particular. The deskewing phase also benefits from the optimisations and shows a reduction in phase runtime of over 98%. In other phases, such as grayscale and the filter, a significant reduction in phase runtime can also be observed, while in the morphological ops phase, a slight increase in phase runtime can be seen. WHY?????. Overall, this leads to a reduction in the total runtime of over 95%. In absolute terms, the runtime improvement is mainly due to the optimisation of Bataineh binarisation, as this phase accounts for the largest share of the total runtime in the baseline implementation.

Table 3 shows the detailed phase runtime for the different phases and images. It can be seen that the optimisations lead to a significant reduction in runtime in all phases, especially in Bataineh binarisation, which accounts for the largest share of the total runtime in the baseline implementation. For larger images such as the 8K text image, it can be seen that the image becomes too large for the system, as there is a sharp increase in the Init & Copy phase, which indicates a memory limitation.

Table 2: Comparison of phase runtimes (3840x2254, 8 Cores)

Phase	baseline (ms)	optimised (ms)	diff (%)
Init & Copy	65,458	63,672	-2,728%
Grayscale	30,950	17,753	-42,640%
Deskew	**159,405**	**3,062**	**-98,079%**
Filters (Sum)	483,312	380,467	-21,279%
Bataineh	**15.507,105**	**217,347**	**-98,598%**
Morphological Ops	66,663	93,05	+39,583%
Color Preservation	4,129	4,089	-0,969%
Total	**16.328,084**	**781,634**	**-95,213%**

5 Future Work

(1) Use heuristics for selecting algorithms, e.g. binarisation
(2) better deskew
(3) runtime experiments on WASM
(4) try to optimise for WASM (reduce the performance gap)
(5) What do we do to avoid false sharing?
(6) Look into blocking code

References

[1] Musab Kasim Alqudah, Mohammad F. Bin Nasrudin, Bilal Bataineh, Mashal Alqudah, and Arwa Alkhatatneh. 2015. Investigation of binarization techniques for unevenly illuminated document images acquired via handheld cameras. In *2015 International Conference on Computer, Communications, and Control Technology (I4CT)*. 524–529. doi:10.1109/I4CT.2015.7219634

[2] Bilal Bataineh, Siti Abdullah, Khairuddin Omar, and Mohammad Faidzul Nasrudin. 2011. Adaptive Thresholding Methods for Documents Image Binarization, Vol. 6718. 230–239. doi:10.1007/978-3-642-21587-2_25

[3] Dan Bloomberg. 2001, 2024. Leptonica Image Processing Library. http://leptonica.org

[4] ImageMagick Studio LLC. 2024. *ImageMagick*. https://imagemagick.org

[5] Jens Ohrnberger. 2022. unpaper: Post-processing tool for scanned sheets. https://github.com/unpaper/unpaper

[6] Nobuyuki Otsu. 1979. A Threshold Selection Method from Gray-Level Histograms. *IEEE Transactions on Systems, Man, and Cybernetics* 9, 1 (1979), 62–66. doi:10.1109/TSMC.1979.4310076

[7] Jean-Philippe Tarel. 2024. CImg: A Simple C++ Toolkit for Image Processing. https://cimg.eu. Version 3.x.

[8] Yanxi Zhou, Shikai Zuo, Zhengxian Yang, Jinlong He, Jianwen Shi, and Rui Zhang. 2023. A Review of Document Image Enhancement Based on Document Degradation Problem. *Applied Sciences* 13, 13 (2023). doi:10.3390/app13137855

Table 3: Detailed runtimes for certain pipeline stages

Phase	1K Small	2K Text	2K Mag.	4K Text	8K Text
Init & Copy	1,29	4,20	61,83	65,46	241,93
Grayscale	0,60	2,95	27,68	30,95	92,45
Deskew	24,22	59,89	28,90	159,41	518,54
Binarisation	71,00	1.555,57	15.999,83	15.507,11	111.495,99
Filter (Sum)	28,74	125,14	520,73	483,31	1.968,43
TOTAL	**130,58**	**1.770,14**	**16.701,17**	**16.328,08**	**114.607,07**